Guia de Publicação — Suprimentos Dream

Projeto: Comparador de cotações Suprimentos Dream (Node.js + Express + OpenAI) **Repositório:** https://github.com/TecnologiaDreamFactory/suprimentos-dream **URL em produção:** https://suprimentos-dream-production.up.railway.app **Status:** EM PRODUÇÃO **Responsável técnico:** Almir Almeida (conta GitHub: `TecnologiaDreamFactory`) **Última atualização do guia:** 21/04/2026

Sumário

1. Visão geral do fluxo
2. Pré-requisitos
3. Etapa 1 — Preparar o projeto local
4. Etapa 2 — Autenticar no GitHub via CLI
5. Etapa 3 — Criar repositório no GitHub e enviar o código
6. Etapa 4 — Deploy no Railway
7. Etapa 5 — Pós-deploy e operação
8. Tabela de custos
9. Tabela de tempos
10. Alertas de segurança
11. Troubleshooting

---

1. Visão geral do fluxo

```
PC local (código) ──(git push)──> GitHub (suprimentos-dream) ──(webhook)──> Railway ──> URL pública
```

1. Código preparado e versionado no PC com `git`.
2. Publicado no GitHub (serve como fonte única de verdade e histórico).
3. Railway conecta no repositório, instala dependências, roda `node server.js` e expõe uma URL pública.
4. Cada `git push` futuro no branch `main` dispara redeploy automático no Railway.

---

2. Pré-requisitos

Item	Verificar se tem	Link para instalar
Git (v2+)	`git --version`	https://git-scm.com/download/win
Node.js (v18+)	`node --version`	https://nodejs.org
GitHub CLI (`gh`)	`gh --version`	https://cli.github.com
Conta no GitHub	Acesse https://github.com	—
Conta na OpenAI + chave de API	https://platform.openai.com/api-keys	—
Conta no Railway	https://railway.com (login recomendado com GitHub)	—

No ambiente usado para este deploy, as versões confirmadas foram:

- Git **2.53.0.windows.1**
- GitHub CLI **2.89.0**
- Node.js (já presente no projeto; requerido pelas dependências em `package.json`)

Etapa 1 — Preparar o projeto local

Status: [CONCLUÍDO] **Tempo gasto:** ~2 minutos **Custo:** R$ 0,00

O que foi verificado

- Arquivo `.gitignore` já protege itens sensíveis:
  - `.env` e variantes (chaves de API)
  - `node_modules/`
  - `rfq/data/batch-history/` e `rfq/data/batch-snapshots/` (dados de runtime)
  - Logs, arquivos de OS/editor
- Arquivo `.env.example` presente como referência de variáveis (sem valores reais).
- `package.json` com o script `"start": "node server.js"` (o Railway usa esse script por padrão).
- `server.js` lê `process.env.PORT` e escuta em `0.0.0.0` (compatível com Railway).

Comandos executados

```
# Na raiz do projeto
git init -b main
git add .
git commit -m "chore: commit inicial do projeto Suprimentos Dream"
```

Resultado: 98 arquivos versionados. `.env` e `node_modules/` ficaram de fora corretamente.

Observação sobre identidade do commit

Para que os commits apareçam vinculados ao perfil correto no GitHub, configure o git com o **mesmo e-mail cadastrado na conta GitHub**:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@github.com"
```

Se precisar reescrever o autor do commit inicial:

```
git commit --amend --reset-author --no-edit
```

Etapa 2 — Autenticar no GitHub via CLI

Status: [CONCLUÍDO] **Tempo gasto:** ~2 minutos **Custo:** R$ 0,00

Comando executado

```
gh auth login
```

Respostas selecionadas no prompt interativo

Pergunta	Resposta
What account do you want to log into?	**GitHub.com**
What is your preferred protocol for Git operations?	**HTTPS**
Authenticate Git with your GitHub credentials?	**Yes**
How would you like to authenticate?	**Login with a web browser**

O CLI exibiu um código de 8 dígitos, abriu o navegador em `https://github.com/login/device`, o código foi colado e a autorização concluída.

Verificação

```
gh auth status
```

Saída (confirmada):

```
github.com
  ✓ Logged in to github.com account TecnologiaDreamFactory (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

Etapa 3 — Criar repositório no GitHub e enviar o código

Status: [CONCLUÍDO] **Tempo gasto:** ~5 segundos (push + criação) **Custo:** R$ 0,00

Comando executado

```
gh repo create suprimentos-dream `
  --public `
  --source=. `
  --remote=origin `
  --description "Comparador de cotacoes Suprimentos Dream (Node/Express + OpenAI)" `
  --push
```

Parâmetros escolhidos:

- Nome: `suprimentos-dream`
- Visibilidade: **pública**
- `--source=.` aponta para a pasta atual
- `--remote=origin` configura o remoto padrão
- `--push` já empurra o branch `main` para o GitHub

Resultado

Repositório criado e sincronizado: https://github.com/TecnologiaDreamFactory/suprimentos-dream

Branch local `main` já rastreando `origin/main`.

Etapa 4 — Deploy no Railway

Status: [CONCLUÍDO] **Data da publicação:** 21/04/2026 **URL pública:** https://suprimentos-dream-production.up.railway.app **Tempo real gasto:** ~12 minutos (criar projeto + colar variáveis + gerar domínio + aguardar build) **Custo:** ver seção 8 (plano Free tem US$ 5 de crédito único; Hobby custa US$ 5/mês)

O Railway não pode ser automatizado aqui porque exige login interativo via navegador. O passo a passo abaixo é feito pela interface web do Railway.

4.1. Criar conta e projeto

1. Acessar https://railway.com
2. Clicar em **Login** → **Login with GitHub**
3. Autorizar o aplicativo Railway a ler seus repositórios
4. Clicar em **New Project** → **Deploy from GitHub repo**
5. Selecionar **TecnologiaDreamFactory/suprimentos-dream**
6. O Railway detecta automaticamente que é um projeto Node.js (via `package.json`) e inicia o primeiro build

> O primeiro build pode falhar por falta de variáveis de ambiente. É esperado. Siga para 4.2.

4.2. Configurar variáveis de ambiente

Dentro do projeto recém-criado:

1. Clicar no serviço **suprimentos-dream**
2. Abrir a aba **Variables**
3. Clicar em **Raw Editor** e colar o bloco abaixo

```
OPENAI_API_KEY=cole-a-nova-chave-aqui
OPENAI_MODEL=gpt-4o-mini
OPENAI_ENABLE_AMBIGUITY_RESOLUTION=true
OPENAI_ENABLE_ANALYTIC_SUMMARY=true
OPENAI_PARSING_CONFIDENCE_THRESHOLD=0.75
OPENAI_TIMEOUT_MS=30000
OPENAI_MAX_RETRIES=1
OPENAI_TEMPERATURE=0.2
OPENAI_ENABLE_SEMANTIC_ITEM_MATCH=true
OPENAI_SEMANTIC_ITEM_MATCH_THRESHOLD=0.80
OPENAI_SEMANTIC_ITEM_MATCH_LOW_CONFIDENCE=0.60
OPENAI_SEMANTIC_ITEM_MATCH_MAX_CANDIDATES=5
OPENAI_SEMANTIC_ITEM_MATCH_MAX_CALLS=10
NODE_ENV=production
```

4. Clicar em **Update Variables**. Um redeploy é disparado automaticamente.

> **IMPORTANTE:** Não defina `PORT` manualmente. O Railway injeta essa variável sozinho. O `server.js` já lê `process.env.PORT` corretamente.

4.3. Gerar domínio público

1. Ainda no serviço, abrir **Settings** → **Networking** → **Public Networking**
2. Clicar em **Generate Domain**
3. Uma URL do tipo `suprimentos-dream-production.up.railway.app` será criada
4. Em **Target Port**, deixar em branco (Railway detecta pela `PORT` injetada)

4.4. Acompanhar o deploy

1. Abrir a aba **Deployments** → **View Logs**
2. Aguardar o log exibir:

```
🚀 SUPRIMENTOS DREAM — comparador de cotações
📡 Backend:     http://localhost:PORT
```

3. Acessar a URL pública gerada. A aplicação deve responder.

4.5. Resultado

- **URL pública:** https://suprimentos-dream-production.up.railway.app
- **Data da primeira publicação:** 21/04/2026
- **Variáveis configuradas:** 14 (13 `OPENAI_*` + `NODE_ENV=production`)
- **Status do serviço:** Active (log exibiu `🚀 SUPRIMENTOS DREAM — comparador de cotações` e `🔑 API OpenAI: ✅ Configurada`)
- **Custo até o momento:** consumindo crédito do Trial (US$ 5 inicial); migração para Hobby (US$ 5/mês) recomendada antes de esgotar

---

Etapa 5 — Pós-deploy e operação

Status: [ATIVO — aplicação em produção]

5.1. Fluxo para publicar atualizações

Qualquer mudança no código só precisa de um push:

```
git add .
git commit -m "descrição curta da mudança"
git push
```

O Railway detecta o push em `main` e redeploy automático é disparado.

5.2. Persistência de dados (atenção)

Por padrão, o filesystem do container do Railway é **efêmero** — a cada redeploy ou reinício, os dados locais são apagados. Isso afeta:

- `rfq/data/batch-history/` — histórico de lotes
- `rfq/data/batch-snapshots/` — snapshots de lote
- XLSX temporários gerados em `/downloads/...`

Opções para persistir:

Opção	Esforço	Custo	Quando usar
Volume do Railway (aba *Settings* → *Volumes*) montado em `/app/rfq/data`	Baixo	Parte do plano	Histórico simples, pequeno volume
Adapter de Object Storage (S3, R2) já presente em `rfq/batch/storage/objectBatchExportAdapter.js`	Médio (configuração)	Pago por uso (centavos/mês)	Produção real com múltiplas réplicas

Para o primeiro deploy, o volume do Railway é suficiente.

5.3. Monitoramento básico

- **Logs em tempo real:** aba *Deployments* → *View Logs* no Railway
- **Métricas de CPU/RAM:** aba *Metrics* no serviço
- **Consumo de crédito:** menu superior → *Usage*

5.4. Reiniciar manualmente

Serviço → menu de três pontos → **Restart**. Útil quando variáveis foram alteradas ou para limpar cache em memória.

---

8. Tabela de custos

Valores de referência em **abril/2026**. Confirmar sempre na página oficial de cada serviço.

Serviço	Plano	Custo	Observações
GitHub	Free (repositório público)	**US$ 0,00**	Repositórios públicos ilimitados.
GitHub	Pro (se migrar para privado)	US$ 4,00/mês	Só necessário se o repo virar privado com recursos avançados.
Railway	Trial	**US$ 5,00 de crédito único**	Suficiente para testes iniciais; expira ao esgotar.
Railway	Hobby	**US$ 5,00/mês**	Inclui US$ 5 de uso. Hibernação desligada. Recomendado para produção leve.
Railway	Pro	US$ 20,00/mês	Para times e maior disponibilidade.
OpenAI API	Pay-as-you-go (`gpt-4o-mini`)	**~US$ 0,15 / 1 M tokens de entrada** e **US$ 0,60 / 1 M de saída**	Custo real do projeto depende do volume. Em uso moderado (dezenas de cotações/dia), geralmente abaixo de US$ 5/mês.

Estimativa mensal realista do projeto em produção leve:

- Railway Hobby: US$ 5,00
- OpenAI API: US$ 1,00–5,00
- GitHub: US$ 0,00
- **Total estimado: US$ 6,00 a US$ 10,00 por mês** (≈ R$ 35,00 a R$ 60,00 com dólar em R$ 6,00)

---

9. Tabela de tempos

Tempo real gasto na execução inicial + estimativa para execuções futuras.

Etapa	Tempo inicial	Tempo em redeploys
1. Preparar projeto	~2 min (conferência de `.gitignore` e commit)	—
2. Login `gh auth login`	~2 min	Nunca mais
3. `gh repo create --push`	~5 seg	—
4. Criar projeto no Railway + ligar GitHub	~3 min	Nunca mais
4.2. Colar variáveis de ambiente	~2 min	Só quando mudar
4.3. Gerar domínio	~30 seg	Nunca mais
4.4. Aguardar primeiro build	~3–5 min	~2–3 min por push
Total primeira publicação	**~15–20 min**	—
Total atualizações futuras (`git push`)	—	**~2–3 min**

---

10. Alertas de segurança

10.1. Chave OpenAI

A chave atual no `.env` local **não foi enviada ao GitHub** (protegida pelo `.gitignore`). Ainda assim, como medida preventiva de rotina, **recomenda-se rotacionar a chave** antes do deploy:

1. Acessar https://platform.openai.com/api-keys
2. Revogar a chave atual
3. Criar uma nova com permissão apenas para os modelos necessários (ex.: `gpt-4o-mini`)
4. Colocar a nova chave no `.env` local **e** nas variáveis do Railway (passo 4.2)

10.2. Nunca commitar `.env`

O `.gitignore` já cobre isso. Em caso de dúvida antes de um commit:

```
git status
# confirmar que .env NÃO aparece na lista
```

10.3. Visibilidade do repositório

O repositório foi criado **público**. Se em algum momento for necessário incluir configurações sensíveis no próprio código, considerar:

- Tornar o repositório privado em *Settings → Danger Zone → Change visibility*
- Ou manter configs sempre em variáveis de ambiente (prática atual, já correta)

10.4. Proteção do branch `main`

Após o projeto em produção, ativar no GitHub:

- *Settings* → *Branches* → *Add rule* → exigir pull request antes de mergear em `main`
- Evita pushes diretos acidentais que quebrem a produção

---

11. Troubleshooting

Sintoma	Causa provável	Como resolver
`gh: command not found`	GitHub CLI não instalado	https://cli.github.com/
`fatal: not a git repository`	Rodando `git` fora da pasta do projeto	`cd` para a pasta correta e tentar de novo
Push rejeitado com `403`	Token do `gh` sem escopo `repo`	`gh auth refresh -s repo`
Build do Railway falha com `Cannot find module`	`node_modules/` commitado ou `package-lock.json` dessincronizado	Conferir `.gitignore`; rodar `npm install` local e commitar o novo `package-lock.json`
Deploy sobe mas app retorna 502	Serviço escutando em porta errada	Conferir que `server.js` usa `process.env.PORT`. Já está correto no projeto.
OpenAI retorna 401	`OPENAI_API_KEY` ausente ou inválida no Railway	Conferir a variável na aba *Variables* do serviço
Histórico de lotes some após deploy	Filesystem efêmero do Railway	Montar volume conforme seção 5.2
Domínio gerado não responde	Primeiro build ainda rodando	Acompanhar *Deployments* → *View Logs* até ver `🚀 SUPRIMENTOS DREAM`

Histórico de revisões do guia

Data/Hora	Etapa atualizada	Observação
21/04/2026	Criação inicial	Etapas 1–3 concluídas; Etapa 4 documentada, aguardando execução
21/04/2026	Etapa 4 concluída	Deploy no Railway efetivado. URL pública gerada: `suprimentos-dream-production.up.railway.app`. Aplicação respondendo em produção.